Audio Equalizer Report

Authors:

Elias Dahl – Abstract, Design, Discussion / Conclusion

Andrés Reis – Introduction, Operation / Testing

Date: 12/11/2025

Contents

1 Abstract **2**

2 Introduction **2**

3 Design **2**

3.1 FIR filter extension 2

3.2 Equalizer module 5

3.3 Driver programs 8

4 Operation / Testing **10**

5 Discussion / Conclusion **14**

6 Appendix **15**

1 Abstract

In this report the authors discuss their design and implementation of a hardware audio equalizer (EQ) peripheral. The EQ utilized prior modules designed during the semester, including a Finite Impulse Response (FIR) filter, AXI4-Lite managers and supporters, and serial peripheral interfaces (SPI), used to communicate with a LTC1865L analog-to-digital Converter (ADC) and LTC1654 digital-to-analog converter (DAC). The FIR filter was updated to support filtering and attenuating 13 frequency bands. This updated filter was tested in a SystemVerilog(SV) testbench before use in the EQ SV module. The EQ module undertook a throughout design and testing cycle until passing simulation tests, at which point it was implemented on a Nexys A7 100-T FPGA. The hardware implementation was driven using a C program running on the A7's microblaze, which initialized the EQ before waiting for communications with a Labview GUI program. The Labview GUI allowed the attenuation factors to be set with sliders before sending them to the C program, which sent the factors to the EQ peripheral and waited for the next set of attenuation factors. The peripheral was used with 3.5mm audio inputs and outputs, allowing audio to be played and filtered though the EQ.

2 Introduction

In this project, we finally implement our final goal of Audio Signal Processing, which consists of a system that is capable of selectively edit frequency bands of an audio signal, by implementing an interface we call Equalizer. It consists of all the projects implemented this semester communicating to make the signal processing possible. For the purposes of the project we will need two SPI modules for the ADC (analog-to-digital converter) and DAC (digital-to-analog converter), One FIR instance which was modified from past projects to be able to accommodate 2 channels and 13 bands. Each band can be independently attenuated (or emphasized) by applying a gain factor to that band's filtered output before recombining the bands into a single output, then 4 axi4lite manager instances to be able to communicate with both SPI's and the FIR.

The report will discuss the design thought process, implementation and results behind the implementation of the project, which include details on how we instantiated/modified each module to ensure functionality. Finally, we discuss the design's quality and give final thoughts about the overall project. Related code used for testing purposes can be found in the appendix.

3 Design

3.1 FIR filter extension

The first part of the design process was extending the FIR filter module to support filtering and attenuating 13 bands of audio. From the project description and in-class discussions, we knew we would need 15 RAMs in the module, 13 for the coefficients of each band and 2 for the sample buffers of the left and right channels. To support this, many signals were extended to be multi-bit or packed. The full module code can be seen in Listing 2 of the Appendix. As for the finite state machine (FSM), the original three states, IDLE, MAC, and ROUND (now ROUND_FIR), would need minor modifications to support multiple bands, and two more states, ATTENUATE and SUM_AND_ROUND, were added to support attenuating each filtered

band. Shown below in Table 1 are the addresses used to read from and write to the filter.

Address	Type	Purpose
'h0 - 'h30	Write	Writes to the next spot in a coefficient RAM. 'h0 ->1st band, 'h30 ->13th band.
'h34 - 'h38	Write	Writes to the next spot in a sample buffer. 'h34 ->L, 'h38 ->R.
'h50	Write	Writes to the macEnable register, allowing the Filter to be turned on or off. 1 ->on, otherwise ->off.
'h3c - 'h40	Read	Reads the filtered result for a channel. 'h3c ->L, 'h40 ->R.

Table 1: Defined addresses and their purposes for the extened FIR filter.

The FSM diagram for the extended FIR filter is shown below in Figure 1.

Figure 1: Updated 13 band FIR filter FSM

Beginning in IDLE, if either sample buffer is written to while the filter is enabled, the filtering process starts. This is done by setting the address into the coefficient RAMs to 0, the address into the buffer RAM to one behind where the last sample was written, and zeroing out the sums for each band and the result for the current channel. Originally, the address into the buffer RAM was set to where the last sample was written, but when using the filter module in our equalizer, an off by one error was occurring causing the samples and coefficients to be multiplied incorrectly. Shifting the address used in the filter FSM back by one solved this issue.

In MAC, the read data of the current channel's sample RAM is multiplied by the read data of each coefficient RAM, and each of these products is added to their respective sum. Then, the address into the coefficient RAMs is incremented, the address into the sample RAM is decremented, and the process is repeated. This happens 279 times, equal to the number of coefficients, before moving to ROUND_FIR.

In ROUND_FIR, the sum of each filtered band is rounded to 1.15, as the multiplications in MAC result in 2.30 sums. The FSM then moves to ATTENUATE.

In ATTENUATE, each rounded sum is multiplied by its corresponding attenuation factor before moving to SUM_AND_ROUND. Each attenuation factor is a 2.14 number, with this point being chosen so the attenuation factor can exactly represent 1. We chose to make the attenuation factors an input into the filter module and have them stored in registers in the main audio equalizer module. This allowed us to update the factors at any time, as if they were stored within the filter module, they would have to be written from the EQ to the filter which would have to interrupt the operation of the filter.

In SUM_AND_ROUND, the attenuated results for each band are summed together, and then the sum is rounded from 3.29 back to 1.15 and stored in the result register for the current channel. The global address into the sample buffer is incremented if the current channel is the right channel so that the next samples are stored in the proper positions. Finally, the current channel is swapped for the next pass before moving back to IDLE.

3.2 Equalizer module

The second part of the design was creating the main audio equalizer module. From the project description and in-class discussions, we knew that our module would need 3 AXI4-Lite managers, for communicating with the DAC SPI, ADC SPI, and FIR filter, as well as an AXI4-Lite supporter for communicating with our testbench and driver program. We also knew from our FIR filter design that we would need registers to store the attenuation factors. Before working on the module itself we wrote a testbench to inform our design, shown in Listing 5 of the Appendix. However, this relatively simple testbench, which only writes the coefficients and clears the sample buffers of the FIR filter before starting the equalizer, left many questions to be answered. We had designed a simple FSM outlining the general process of what the EQ would need to do, but figuring out exactly what order we should do the necessary operations and what operations we needed to wait on would be an iterative process. The module code is shown Listing 4 of the Appendix. Table 2 below shows the write addresses for the EQ module.

Address	Type	Purpose
'h0 - 'h30	Write	Writes to the next spot in a coefficient RAM. Used to fill the FIR coefficient RAMs before running the EQ. 'h0 ->1st band, 'h30 ->13th band.
'h34 - 'h38	Write	Writes to the next spot in a sample buffer. Used to 0 initialize the FIR sample buffers before running the EQ. 'h34 ->L, 'h38 ->R.
'h3c	Write	Writes to the EQEnable register, allowing the equalizer to be turned on. 1 ->on, otherwise ->off.
'h40 - 'h70	Write	Writes to one of the 13 attenuation factor registers. 'h40 ->1st band, 'h70 ->13th band.

Table 2: Defined addresses and their purposes for the audio equalizer module.

Shown below in Figure 2 is a FSM diagram for the audio equalizer peripheral. Note that for packed signals with a number in square brackets, such as M_wrAddr[0], 0 signals interact with the ADC SPI, 1 signals interact with the DAC SPI, and 2 signals interact with the FIR filter.

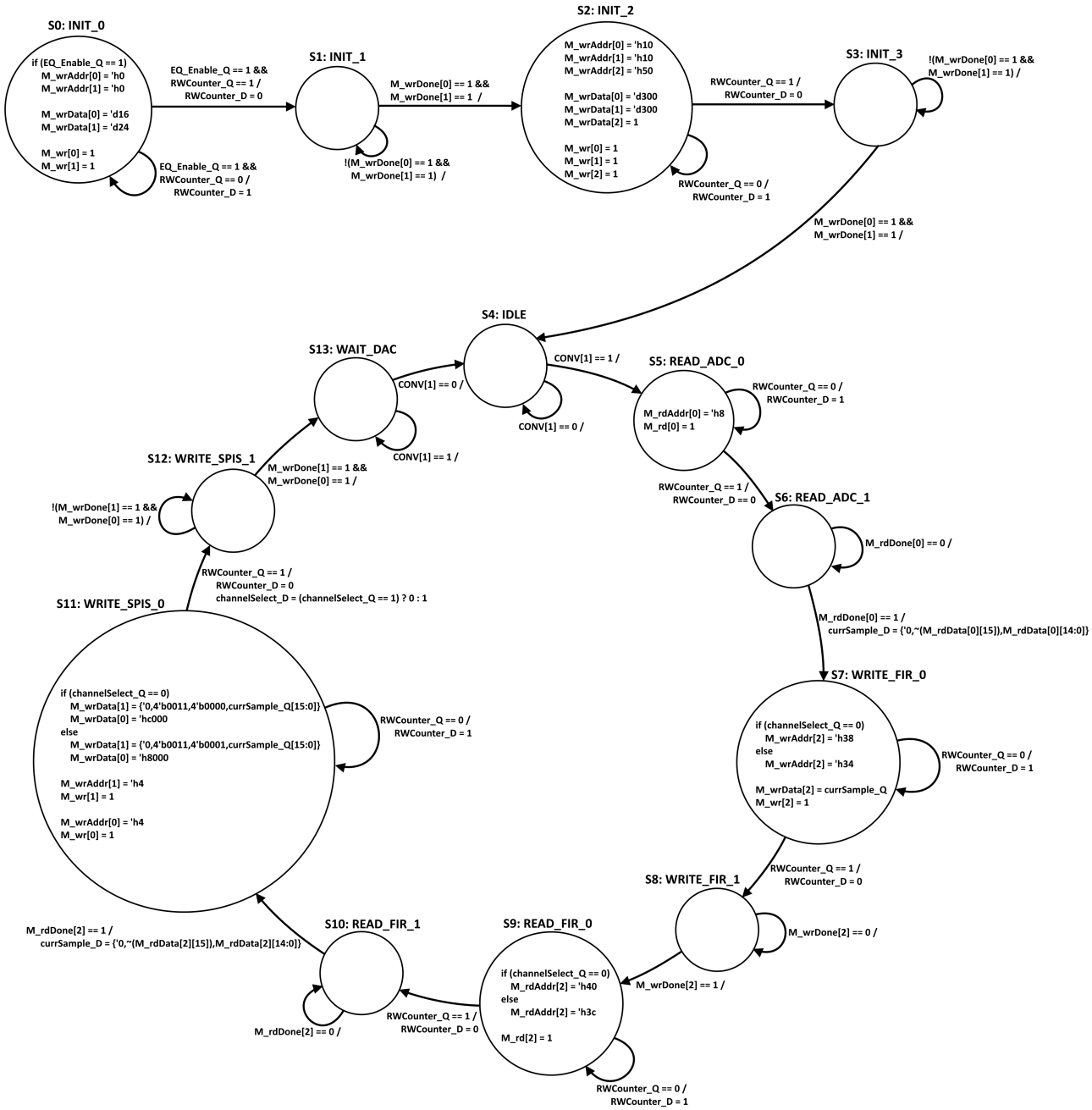

Figure 2: Audio Equalizer FSM

Beginning in INIT_0, the FSM stays in this state until the EQ is enabled. When enabled, INIT_0 initiates a write to both SPIs indicating how many data bits they need to use. The write signals and data are held for 2 cycles to prevent any glitching before moving to INIT_1. INIT_1 waits for these writes to complete before moving to INIT_2.

In INIT_2, writes are initiated to both SPIs indicating how many cycles should occur before bringing CONV low, which determines the sample rate of the ADC and DAC. A write to the FIR filter's enable flop is also initiated. The write signals and data are held for 2 cycles before moving to INIT_3, which waits for the writes to complete before moving to IDLE.

IDLE waits for the DAC SPI's CONV to go high before moving to READ_ADC_0, indicating the DAC finished converting its last sample. This also effectively waits for the ADC and last FIR result to complete as well, as both take less time to complete than the DAC.

READ_ADC_0 initiates a read of the ADC SPI's converted sample. The read signal and address are held for two cycles before moving to READ_ADC_1, which waits for the read to complete before moving to WRITE_FIR_0. As READ_ADC_1 moves to WRITE_FIR_0, the read sample is converted to a signed number for use by the filter and stored in the currSample register.

WRITE_FIR_0 initiates a write to one of the filters two sample buffers depending on the value of the channelSelect register. The write signal, address, and data is held for 2 cycles before moving to WRITE_FIR_1. WRITE_FIR_1 waits for the write to complete before moving to READ_FIR_0.

READ_FIR_0 initiates a read of the filtered sample from the last loop of the FSM from the appropriate result register of the filter. The read signal and address are held for 2 cycles before moving to READ_FIR_1. READ_FIR_1 waits for the read to complete before moving to WRITE_SPIS_0. In this transition, the read sample from the FIR is converted to unsigned and stored in the currSample register.

WRITE_SPIS_0 initiates a write of the sample in currSample, along with its address and control bits, to the DAC SPI. It also initiates a write to the ADC SPI telling it which channel to use next. The address bits written to the DAC SPI and the channel to be used by the ADC SPI are determined by the current channel. The write signals, addresses, and data are held for 2 cycles before moving to WRITE_SPIS_1. In this transition, the current channel is swapped for the next loop of the FSM. WRITE_SPIS_1 waits for the writes to complete before moving to WAIT_DAC.

WAIT_DAC waits for the DAC SPI's CONV to be low before moving to IDLE. This ensures the DAC SPI sample conversion starts so IDLE can properly wait for it to complete.

3.3 Driver programs

After implementing the EQ module in hardware, a C driver program and LabView GUI program were developed for use with the peripheral. The C program, shown in Listing 6, fills the coefficient RAMs and 0 initializes the sample RAMs, and starts the equalizer. It then waits for attenuation factors to be sent from the LabView program, which it sends to the peripheral, and then waits new attenuation factors to be sent from LabView.

The LabView program, which was provided to us, allows attenuation factors to be set using sliders, which are sent to the C program when the send button is pressed. A view of the GUI is shown below in Figure 3.

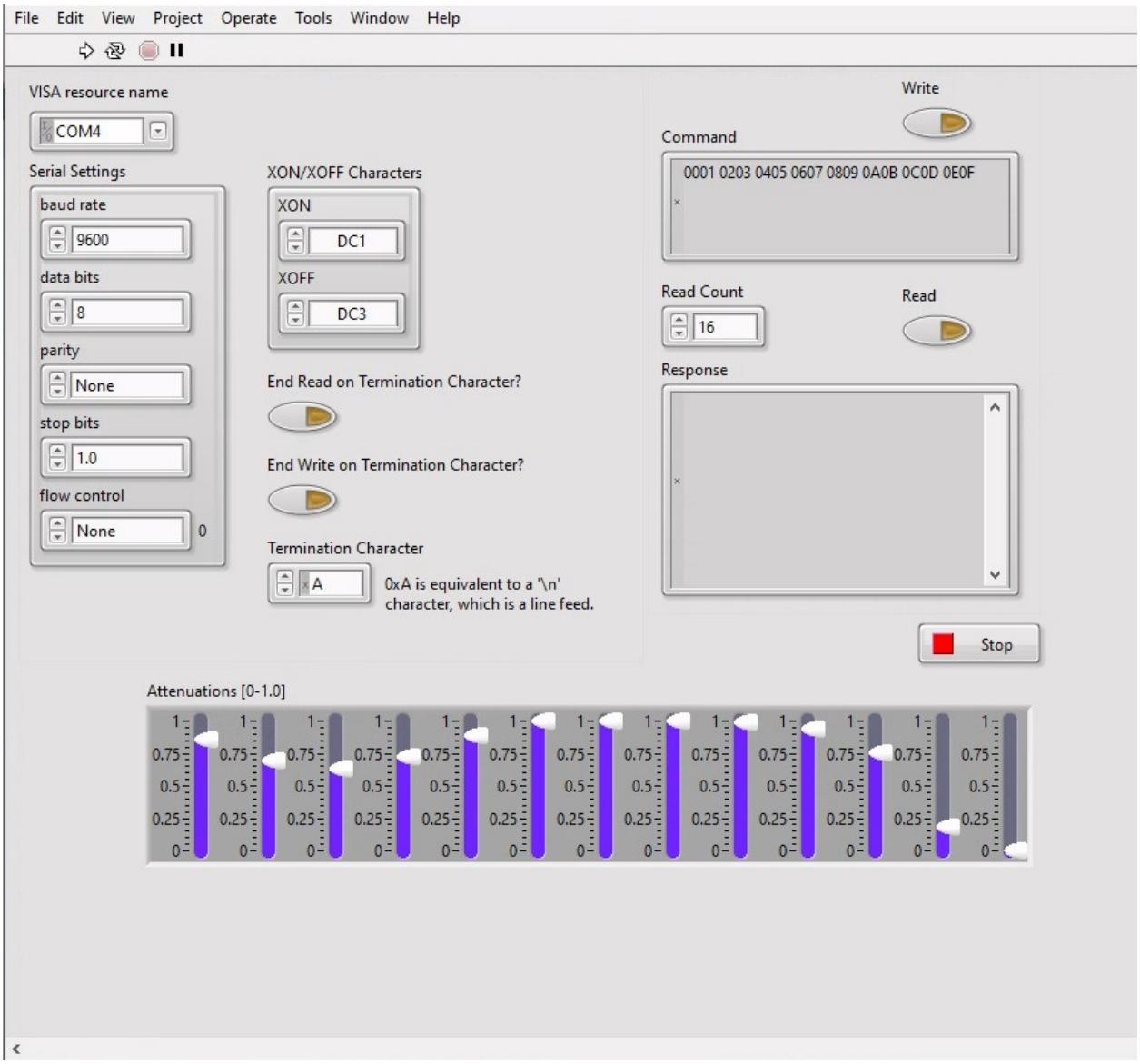

Figure 3: LabView GUI

4 Operation / Testing

The testing phase of the project began by implementing a working FIR model in the C programming language basically creating a software version of how our FIR should work proving our understanding of the concepts needed to implemented in hardware later. In this part of the project, we used basic C concepts such as arrays, pointers, buffers, if statements, and loops to simulate the Equalizer process. And the purpose of recreating it is mainly because we need to accommodate 2 channels and 13 bands for the EQ model to work as intended.

The C model mimics exactly what our final FIR module in Hardware should do. The implementation in simple terms takes input samples as if they were coming from the ADC, runs them through an FIR filter for each band, applies the band's attenuation, and adds everything together to form the signal that would eventually be sent to the DAC.

In order to test our C model we used 2 sinusoidal waves of amplitude 0.9 and frequencies of 1318 Hz and 1734 Hz as input signals. The signals were then processed by our equalizer module on C and printing the output into the console which is then imported to Excel with the input in order to measure the gain of the equalizer at both tested frequencies. The gain was measured by getting the peak to peak amplitude of the input signal and the peak to peak amplitude of the corresponding output signal. It can be then computed as:

$$\text{Gain}=20\log_{10}\left(\frac{A_{\text{out,pp}}}{A_{\text{in,pp}}}\right).$$

Using the frequency response of the equalizer design tool, we were able to obtain the expected gain. We then to ensure our results were correct subtract the expected gain from the measured gain in this way, proving the result matches the expected gain and confirming that our FIR model implementation in C is correct.

Once our C implementation was completed the next step of our testing phase consisted of making sure our testbench implemented in SystemVerilog (SV) worked for the FIR portion of the project.

The FIR (finite impulse response) filter implemented in SystemVerilog works with the same logic as our previous FIR project, however this time the module was modifies in order to accommodate 2 channels and 13 bands.

The testbench was implemented using the same input data and coefficients than the C code in order to prove everything was working as intended on both sides. It starts by initializing all control signals and uses *readmemh* on the left and right channels to input the samples, the FIR coefficients and the attenuation factors all into RAM blocks for use. After the Initialization for each FIR instance j, it sets the coefficient RAM address and then iterates over all taps from NUM TAPS down to 0, writing the coefficients through the Manager. Then sample by sample we write to the buffer addresses and we wait for the multiply and accumulate to complete (MAC) and after they are done we then store them into result registers to be later printed into the console.

Once the results of the FIR testbench are printed into the Vivado console they are inputted into Microsoft Excel in order to confirm functionality. By comparing the measured gain from the FIR output with the measured gain from these outputs to the expected gain from the design tool and the C reference model,

we confirm that the FIR model is working. Below we can see for each test frequency (1318 Hz on the left channel figure 4 and 1734 Hz on the right channel figure 5) the plot shows two traces: one corresponding to the measured output samples from the FIR testbench and the other corresponding to the expected waveform computed from the design-tool gain for that frequency.

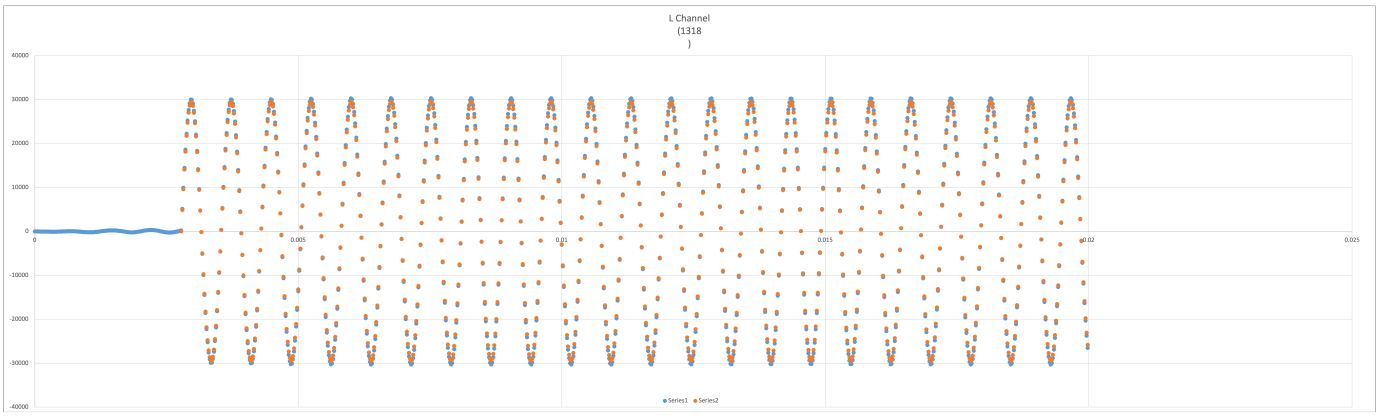

Figure 4: Left Channel 1318 Hz

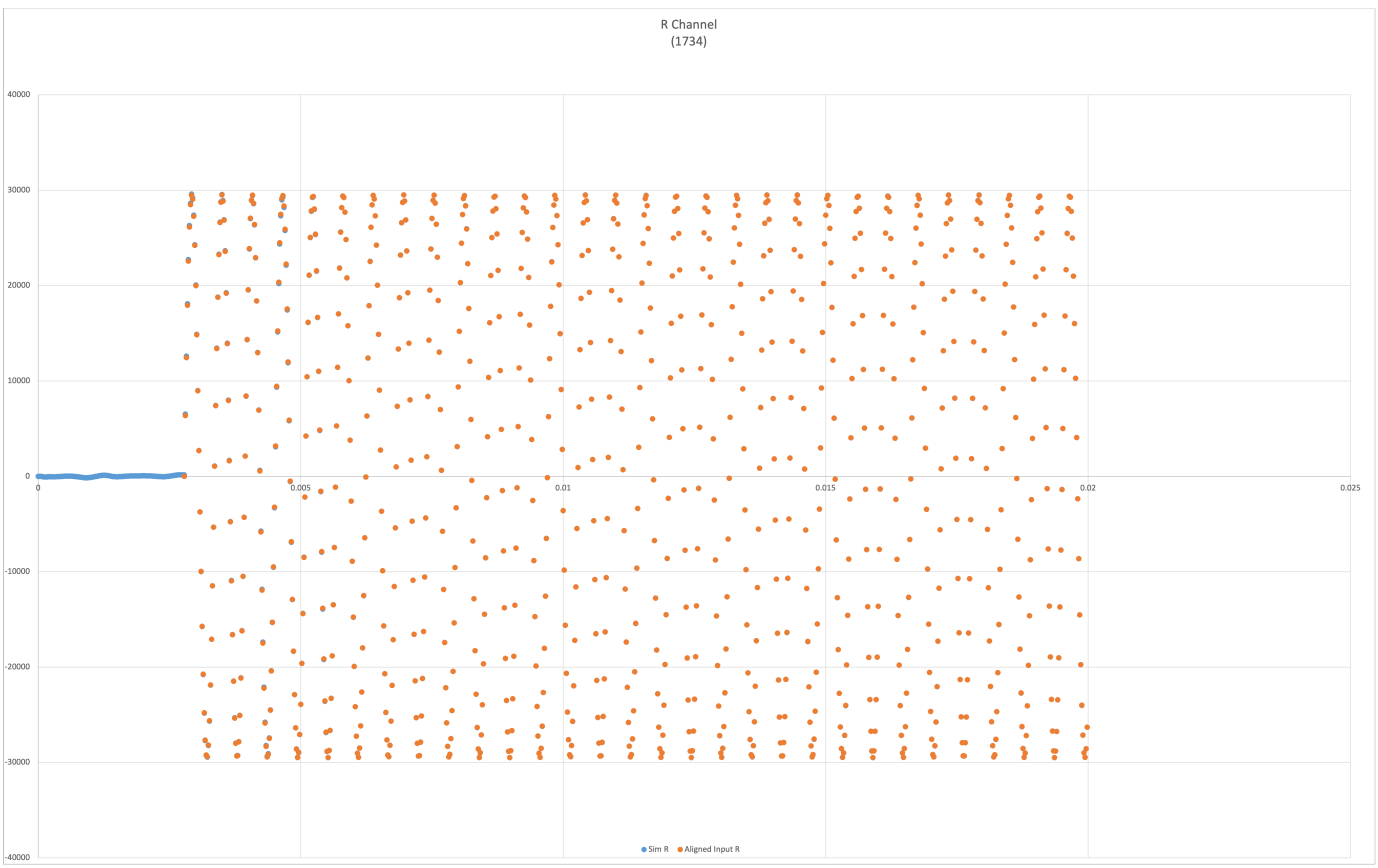

Figure 5: Right Channel 1734 Hz

Once the FIR implementation was completed and verified. It was then time to test the Equalizer module that wraps all the projects we have been working on during the semester. The first Part of the testing for the module consisted of a SystemVerilog Testbench. Which verifies the whole EQ module working properly inside Vivado.

The EQ testbench work consists of Loading the FIR coefficients into a memory file this way loading them into our EQ module for the FIR module to be able to use them. After the coefficients are set, the testbench programs the attenuation registers for each band. For now, every band is given a unity gain value ($0x4000$ in 2.14 fixed-point), which means the equalizer should pass the signal without boosting or cutting any frequency band. Finally, the testbench turns the EQ on by writing 1 to the enable register. At that point, the ADC tester starts sending samples into the EQ, and the DAC tester receives the processed samples on the other SPI interface. This part of the project is the one that took the most debugging due to the attention to detail required to build the EQ FSM to meet every detail. Just like in our other SystemVerilog testbench in Vivado we would print the result waveform into the console and export it using Microsoft Excel to be compared with the expected waveform. The signed 16-bit outputs printed by the EQ testbench align with the C reference outputs (C 1734 and C 1318), with the difference columns equal to zero over the main steady-state region. This shows that the EQ model implemented in SystemVerilog is working just like our C model.

We tested the Equalizer implementation outside of Vivado by doing 2 more things: we measured its frequency response on the bench using a function generator and an oscilloscope. Using the function generator we swept the frequency while we measured the peak to peak voltage of the output and recorded the values in Excel as we can see in the table below. By doing this we wanted to compare it against the frequency response generated by the tool. As seen below the results came to be very similar to the tool's frequency response.

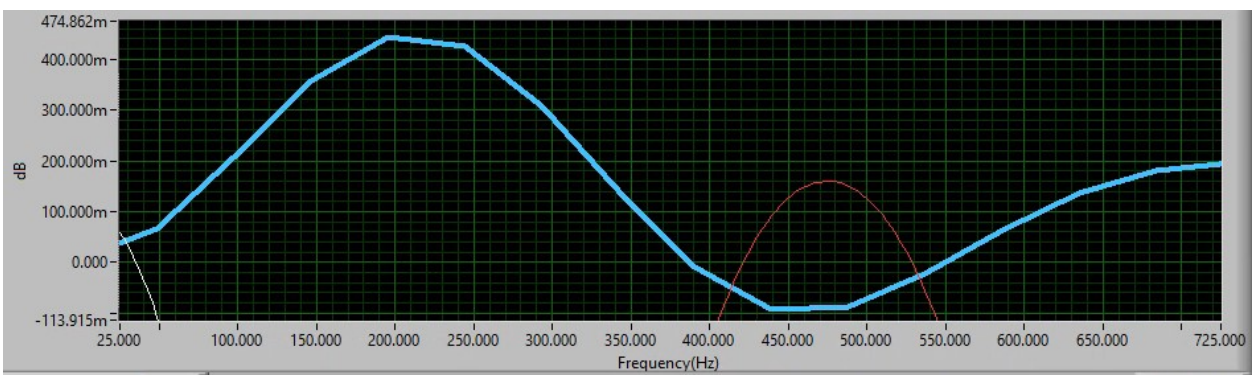

Figure 6: Expected Frequency Response

Figure 7: Frequency Response from Results

Finally by using real time audio. We connected a AUX wire of a laptop/phone to the ADC inputs and played music through the equalizer while also having the capability of modifying/update the attenuation factors in real time. As we boosted and cut different bands, we could clearly hear the expected changes verifying once again our Equalizer project was successful.

5 Discussion / Conclusion

Although we didn't build a full C model of the equalizer and measure it's performance, from the performance speedup seen in the original FIR filter design, approximately 21x, we can conclude that that our hardware equalizer peripheral would be much faster, and thus able to use a higher sample rate, than a software version. Getting this peripheral to work was extremely gratifying for us, however given more time to work on the module, there are some design changes that could have been made.

One large design philosophy that we tried to employ at the beginning but fell apart as the deadline drew closer was making our designs easily extensible. We have the basis of this down, especially in the filter module, using parameters to define how many channels, frequency bands and taps should be used. However, as we had to make quick changes to our design to get it to work, we didn't always adhere to this principle, such as hard-coding addresses rather than using equations based on the parameters.

Another area where our design could have been better in retrospect is operation safety. The biggest instance where improved safety measure would be beneficial are writes. Some writes, such as writing to the coefficient RAMs, probably shouldn't be allowed when the modules are enabled or running, as such a write would ruin the output of the module.

To conclude, our audio equalizer module, along with its GUI and driver programs, allowed us to attenuate 13 bands of an audio signal and output the result, which is super cool! When designing the EQ module, the test bench design wasn't able to inform our design as much as with previous modules, which forced us to take a different approach. The process was much more iterative, testing the module and modifying it repeatedly, ironing errors and refining the design. This isn't completely dissimilar to the testing we've done with other modules, but the scale at which the design changed between testings, as well as from the initial design to the final design, was much greater. Additionally, the thoroughness at which we tested the design in simulation in this process resulted in a design that needed no major changes when implementing in hardware, besides allocating more RAM to the Microblaze, which we forgot to do initially. To be able to take everything we worked on over the semester, expand upon it, and combine it in one final project was challenging, but one of the most rewarding things we've done in our academic career.

6 Appendix

```c
/*****************************************************************************
* Copyright (C) 2023 Advanced Micro Devices, Inc. All Rights Reserved.
* SPDX-License-Identifier: MIT
******************************************************************************/
/*
 * helloworld.c: simple test application
 *
 * This application configures UART 16550 to baud rate 9600.
 * PS7 UART (Zynq) is not initialized by this application, since
 * bootrom/bsp configures it to baud rate 115200
 *
 * ------------------------------------------------
 * | UART TYPE   BAUD RATE                        |
 * ------------------------------------------------
 *   uartns550   9600
 *   uartlite    Configurable only in HW design
 *   ps7_uart    115200 (configured by bootrom/bsp)
 */

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

#include "coeffs.h"
#include "sines.h"

short buffer[NUM_TAPS];
short output[NUM_SAMPLES];
int bufferIndex;

void InitFilter(){
  for(int i = 0;i < NUM_TAPS;++i){
    buffer[i] = 0;
  }
  bufferIndex = 0;
}

short Filter(int band){
  int sum = 0;
  int tap_index = bufferIndex ;

  for(int k = 0; k < NUM_TAPS; k++){
     sum += coeffs[band][k]*buffer[tap_index];
     tap_index-- ;
     if (tap_index < 0){
       tap_index = NUM_TAPS-1 ;
     }
  }
  sum += 0x4000; // 1/2 of LSB.
```

```c
  short roundedSum = sum >> 15;
  return roundedSum;
}

int main()
{
    init_platform();
    xil_printf("START!") ;

    for(int band = 0; band < NUM_BANDS; ++band){
      InitFilter();
      for (int n = 0; n < NUM_SAMPLES; ++n){
          buffer[bufferIndex] = sine_1318[n];
          output[n] += Filter(band);
          bufferIndex++;
          if(bufferIndex > NUM_TAPS-1){
            bufferIndex = 0;
          }
      }
    }


    for(int i = 0; i < NUM_SAMPLES; i++){
      xil_printf("%d\n", output[i]) ;
  }
    cleanup_platform();
    return 0;
}
```

Listing 1: Updated FIR C model.

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////
// Engineers: Elias Dahl, Andres Reis.
// Module Name: FIR_Filter
////////////////////////////////////////////////////////////////////////////////


module FIR_Filter_Updated#( parameter C_S_AXI_DATA_WIDTH = 32, C_S_AXI_ADDR_WIDTH
    = 9, RAM_DATA_WIDTH = 16, RAM_ADDR_WIDTH = 9, FIR_COPIES = 13, NUM_CHANNELS =
    2, NUM_TAPS = 279)
(   input  logic  S_AXI_ACLK,
    input  logic  S_AXI_ARESETN,
    input  logic  [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR,
    input  logic  S_AXI_AWVALID,
    output logic  S_AXI_AWREADY,
    input  logic  [C_S_AXI_DATA_WIDTH-1:0] S_AXI_WDATA,
    input  logic  [3:0] S_AXI_WSTRB,
    input  logic  S_AXI_WVALID,
    output logic  S_AXI_WREADY,
    input  logic  [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_ARADDR,
    input  logic  S_AXI_ARVALID,
    output logic  S_AXI_ARREADY,
    output logic  [C_S_AXI_DATA_WIDTH-1:0] S_AXI_RDATA,
    output logic  [1:0] S_AXI_RRESP,
    output logic  S_AXI_RVALID,
    input  logic  S_AXI_RREADY,
    output logic  [1:0] S_AXI_BRESP,
    output logic  S_AXI_BVALID,
    input  logic  S_AXI_BREADY,
    output logic JA_1,
    input logic signed [C_S_AXI_DATA_WIDTH-1:0] attenFactors[0:FIR_COPIES-1] //
        Uses only 16 bits, could be shortend.
    );

    // coeff ram sigs
    logic  [RAM_DATA_WIDTH-1:0] ram_coeff_wdata;
    logic  [RAM_ADDR_WIDTH-1:0] ram_coeff_addr;
    logic  [FIR_COPIES-1:0] wr_coeff;
    logic signed  [RAM_DATA_WIDTH-1:0] ram_coeff_rdata[0:FIR_COPIES-1] ;

    // buffer ram sigs
    logic  [NUM_CHANNELS-1:0]wr_buff;
    logic  [RAM_ADDR_WIDTH-1:0] ram_buff_addr [0:NUM_CHANNELS-1] ;
    logic  [RAM_DATA_WIDTH-1:0] ram_buff_wdata ;
```

```
logic signed [RAM_DATA_WIDTH-1:0] ram_buff_rdata[0:NUM_CHANNELS-1] ;

logic   [C_S_AXI_ADDR_WIDTH-1:0]    wrAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    wrData ;
logic                               wr ;
logic   [C_S_AXI_ADDR_WIDTH-1:0]    rdAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    rdData ;
logic                               rd ;

logic signed    [C_S_AXI_DATA_WIDTH-1:0] sum_D[0:FIR_COPIES - 1], sum_Q[0:
    FIR_COPIES - 1];
logic signed    [C_S_AXI_DATA_WIDTH-1:0] result_D[0:NUM_CHANNELS-1], result_Q
    [0:NUM_CHANNELS-1];

logic           [RAM_ADDR_WIDTH-1:0] kAddr_D, kAddr_Q;
logic           [RAM_ADDR_WIDTH-1:0] tapAddr_D, tapAddr_Q;
logic           [RAM_ADDR_WIDTH-1:0] buffAddr_D, buffAddr_Q;

logic                                channelSelect_D, channelSelect_Q;

logic [C_S_AXI_DATA_WIDTH-1:0] macStatus;

logic macEnable_D, macEnable_Q;

// Used for generates.
genvar j;

// FSM state
typedef enum logic [2:0] {IDLE, MAC, ROUND_FIR, ATTENUATE, SUM_AND_ROUND}
    statetype;
statetype nextState, currState;

// Sequential Logic
always_ff @ (posedge S_AXI_ACLK) begin
    if(!S_AXI_ARESETN) begin
        for(int i = 0; i < FIR_COPIES; i++) begin
            sum_Q[i] <= 0;
        end
        for(int i = 0; i < NUM_CHANNELS; i++) begin
            result_Q[i] <= 0;
        end
        kAddr_Q <= 0;
        tapAddr_Q <= 0;
        buffAddr_Q <= 0;
```

```
            channelSelect_Q <= 0;
            macEnable_Q <= 0;
            currState <= IDLE;
        end
          else begin
            for(int i = 0; i < FIR_COPIES; i++) begin
                sum_Q[i] <= sum_D[i];
            end
            for(int i = 0; i < NUM_CHANNELS; i++) begin
                result_Q[i] <= result_D[i];
            end
            currState <= nextState;
            kAddr_Q <= kAddr_D;
            tapAddr_Q <= tapAddr_D;
            buffAddr_Q <= buffAddr_D;
            macEnable_Q <= macEnable_D;
            channelSelect_Q <= channelSelect_D;
          end
    end

    always_comb begin
        // default signal values.
        for(int i = 0; i < FIR_COPIES; i++) begin
            sum_D[i] = sum_Q[i];
        end
        for(int i = 0; i < NUM_CHANNELS; i++) begin
            result_D[i] = result_Q[i];
        end
        kAddr_D = kAddr_Q;
        tapAddr_D = tapAddr_Q;
        buffAddr_D = buffAddr_Q;
        channelSelect_D = channelSelect_Q;
        macEnable_D = macEnable_Q;
        rdData = 0;
        macStatus = 0;
        ram_coeff_wdata = 0;
        wr_coeff = 0;
        ram_buff_wdata = 0;
        wr_buff = 0;
        nextState = currState;
        ram_coeff_addr = 0;
        ram_buff_addr[0] = 0;
        ram_buff_addr[1] = 0;
        JA_1 = 0;
```

```
case(currState)
    // Stay in IDLE and do nothing unless a sample
    // is written to the buffer.
    // If a sample is written, intialize the registers
    // used when MACing, and move to MAC.
    IDLE: begin
        if(wr && (wrAddr == 'h34 || wrAddr == 'h38) && macEnable_Q == 1)
            begin
            macStatus = 1;
            kAddr_D = 0;
            /*
            * Added the If statement below to solve an off by one error
                that occurs
            * when this module is used in the Audio Equalizer.
            */
            if(buffAddr_Q == 0) begin
                tapAddr_D = NUM_TAPS-1;
            end
            else begin
                tapAddr_D = buffAddr_Q-1;
            end

            //tapAddr_D = buffAddr_Q;
            for(int i = 0; i < FIR_COPIES; i++) begin
                sum_D[i] = 0;
            end
            result_D[channelSelect_Q] = 0; //
            nextState =  MAC;
        end
    end

    // Repeatedly MAC until all samples in the buffer
    // have been used, then proceed to ROUND.
    MAC: begin
        macStatus = 1;
        JA_1 = macStatus;
        ram_coeff_addr = kAddr_Q;
        ram_buff_addr[channelSelect_Q] = tapAddr_Q;

        for(int i = 0; i < FIR_COPIES; i++) begin
            sum_D[i] = sum_Q[i] + (ram_coeff_rdata[i]* ram_buff_rdata[
                channelSelect_Q]);
        end
```

```
        if(kAddr_Q == NUM_TAPS-1) begin
            nextState = ROUND_FIR;
        end
        else begin
            kAddr_D = kAddr_Q + 1;
            if(tapAddr_Q == 0) begin
                tapAddr_D = NUM_TAPS-1; //Num taps -1
            end
            else begin
                tapAddr_D = tapAddr_Q - 1;
            end
            nextState = MAC;
        end
    end

    // Add half of the LSB and then effectively
    // arithmetic right shift 15 bits to round the result to Q1.15.
    ROUND_FIR: begin
        macStatus = 1;
        JA_1 = macStatus;
        for(int i = 0; i < FIR_COPIES; i++) begin
            sum_D[i] = (sum_Q[i] + 'h4000);
            sum_D[i] = {{16{sum_D[i][30]}},sum_D[i][30:15]}; // result >>>
                 15
        end


        nextState = ATTENUATE;
    end

    /* Multiply each band sum by its corresponding attenuation factor. */
    ATTENUATE: begin
        JA_1 = macStatus;
        for(int i = 0; i < FIR_COPIES; i++) begin
            sum_D[i] = sum_Q[i]*attenFactors[i];
        end
        nextState = SUM_AND_ROUND;
    end

    /* Add the result for each attenuated band */
    SUM_AND_ROUND: begin
        JA_1 = macStatus;
        for(int i = 0; i < FIR_COPIES; i++)begin
```

```
                result_D[channelSelect_Q] = result_D[channelSelect_Q]+sum_Q[i
                    ];
            end
            result_D[channelSelect_Q] = result_D[channelSelect_Q] + 'h2000;
            result_D[channelSelect_Q] = {{16{result_D[channelSelect_Q][29]}},
                result_D[channelSelect_Q][29:14]};

            // Update buffAddr after both channels have been filtered.
            if(channelSelect_Q == 1) begin
                if(buffAddr_Q == NUM_TAPS-1)
                    buffAddr_D = 0;
                else
                    buffAddr_D = buffAddr_Q + 1;
            end

            //Swap channel for next pass.
            channelSelect_D = (channelSelect_Q == 1) ? 0 : 1;
            nextState = IDLE;
        end

        default: begin
            nextState = IDLE;
        end
    endcase

// RAM write logic
 if(wr) begin
     // Where data is written to is handled by the filter.
     // The RAMs are acting as circular buffers, with each new write
     // going to the next spot in the buffer, eventually looping around
     // and overwriting old values.
     // Addresses from 0x0 to 0x30 (0-48)
     for(int i=0;i<FIR_COPIES;i++) begin
          if(wrAddr == (4 * i)) begin
              ram_coeff_wdata = wrData;
              ram_coeff_addr = buffAddr_Q;
              wr_coeff[i] = wr;

              if(buffAddr_Q == NUM_TAPS-1) begin
                  buffAddr_D = 0;
              end
              else begin
                  buffAddr_D = buffAddr_Q + 1;
              end
```

```
        end
    end

    // Addresses from 0x34 to 0x38 (52-56)
    for(int i=0; i < NUM_CHANNELS; i++) begin
     if(wrAddr == 4 * FIR_COPIES + 4 * i) begin
         ram_buff_wdata = wrData;
         ram_buff_addr[i] = buffAddr_Q;
         wr_buff[i] = wr ;
     end
    end

    /* Used to enable/disable the filter.*/
    if(wrAddr == 'h50) begin
         macEnable_D = wrData[0];
    end


end


// RAM read logic
// Addresses begin at 0x3c (60)
if (rd) begin
    case(rdAddr)
        // Read left channel result
        'h3c: begin
            rdData = result_Q[0];
        end
        //Read right channel result
        'h40: begin
            rdData = result_Q[1];
        end
        // Check if the filter is MACing / not IDLE
        'h44: begin
            rdData = macStatus;
        end
        // Default case, useful for testing.
        // This value will be seen if we are reading data but incorrectly
            using addresses.
        default: begin
            rdData = 'hdeadbeef;
        end
    endcase
```

```
        end
    end

// RAMs for holding the filter coefficients.
generate
    for (j = 0; j < FIR_COPIES; j++) begin
        ram #(.RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),.RAM_DATA_WIDTH(RAM_DATA_WIDTH))
            ram_coeffs (
            .wr(wr_coeff[j]),                       // input
            .DRAM_ACLK(S_AXI_ACLK),             // input
            .DRAM_RESETN(S_AXI_ARESETN),        // input
            .DRAM_ADDR(ram_coeff_addr),         // input
            .DRAM_WDATA(ram_coeff_wdata),       // input
            .DRAM_RDATA(ram_coeff_rdata[j])         // output
            ) ;
    end
endgenerate

// RAMs for holding the samples.
generate
    for (j = 0; j < NUM_CHANNELS; j++) begin
ram #(.RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),.RAM_DATA_WIDTH(RAM_DATA_WIDTH)) ram_buffer
    (
    .wr(wr_buff[j]),                        // input
    .DRAM_ACLK(S_AXI_ACLK),             // input
    .DRAM_RESETN(S_AXI_ARESETN),        // input
    .DRAM_ADDR(ram_buff_addr[j]),           // input
    .DRAM_WDATA(ram_buff_wdata),        // input
    .DRAM_RDATA(ram_buff_rdata[j])          // output
    ) ;
    end
endgenerate

// Axi4Lite Supporter Bus, used for reading and writing data to the Filter RAM and
     registers.
Axi4LiteSupporter #(.C_S_AXI_ADDR_WIDTH(C_S_AXI_ADDR_WIDTH),.C_S_AXI_DATA_WIDTH(
    C_S_AXI_DATA_WIDTH)) Axi4LiteSupporter1 (
    // Simple Bus
    .wrAddr(wrAddr),                    // output   [C_S_AXI_ADDR_WIDTH-1:0]
    .wrData(wrData),                    // output   [C_S_AXI_DATA_WIDTH-1:0]
    .wr(wr),                            // output
    .rdAddr(rdAddr),                    // output   [C_S_AXI_ADDR_WIDTH-1:0]
    .rdData(rdData),                    // input    [C_S_AXI_ADDR_WIDTH-1:0]
    .rd(rd),                            // output
```

```
    // Axi4Lite Bus
    .S_AXI_ACLK(S_AXI_ACLK),            // input
    .S_AXI_ARESETN(S_AXI_ARESETN),      // input
    .S_AXI_AWADDR(S_AXI_AWADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
    .S_AXI_AWVALID(S_AXI_AWVALID),      // input
    .S_AXI_AWREADY(S_AXI_AWREADY),      // output
    .S_AXI_WDATA(S_AXI_WDATA),          // input    [C_S_AXI_DATA_WIDTH-1:0]
    .S_AXI_WSTRB(S_AXI_WSTRB),          // input    [3:0]
    .S_AXI_WVALID(S_AXI_WVALID),        // input
    .S_AXI_WREADY(S_AXI_WREADY),        // output
    .S_AXI_ARADDR(S_AXI_ARADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
    .S_AXI_ARVALID(S_AXI_ARVALID),      // input
    .S_AXI_ARREADY(S_AXI_ARREADY),      // output
    .S_AXI_RDATA(S_AXI_RDATA),          // output   [C_S_AXI_DATA_WIDTH-1:0]
    .S_AXI_RRESP(S_AXI_RRESP),          // output   [1:0]
    .S_AXI_RVALID(S_AXI_RVALID),        // output
    .S_AXI_RREADY(S_AXI_RREADY),        // input
    .S_AXI_BRESP(S_AXI_BRESP),          // output   [1:0]
    .S_AXI_BVALID(S_AXI_BVALID),        // output
    .S_AXI_BREADY(S_AXI_BREADY)         // input
    ) ;

endmodule
```

Listing 2: Updated FIR SV module.

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Engineer: Elias Dahl, Andres Reis
//
// Create Date: 10/02/2025 01:30:37 PM
// Design Name:
// Module Name: FIR_Filter_TB
// Project Name:
// Target Devices:
// Description:
//
// Dependencies:
//
//////////////////////////////////////////////////////////////////////////////////


module FIR_Filter_TB();
parameter C_S_AXI_DATA_WIDTH = 32, C_S_AXI_ADDR_WIDTH = 9, CLK_PERIOD = 33.33,
    RAM_DATA_WIDTH = 16, RAM_ADDR_WIDTH = 9, NUM_TAPS = 279, NUM_SAMPLES = 1000,
    FIR_COPIES = 13;
// Axi4Lite signals
logic   S_AXI_ACLK ;
logic   S_AXI_ARESETN ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR ;
logic   S_AXI_AWVALID ;
logic   S_AXI_AWREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] S_AXI_WDATA ;
logic   [3:0] S_AXI_WSTRB ;
logic   S_AXI_WVALID ;
logic   S_AXI_WREADY ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_ARADDR ;
logic   S_AXI_ARVALID ;
logic   S_AXI_ARREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] S_AXI_RDATA ;
logic   [1:0] S_AXI_RRESP ;
logic   S_AXI_RVALID ;
logic   S_AXI_RREADY ;
logic   [1:0] S_AXI_BRESP ;
logic   S_AXI_BVALID ;
logic   S_AXI_BREADY ;

// Manager simple Bus signals
logic   [C_S_AXI_ADDR_WIDTH-1:0]    wrAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    wrData ;
```

```
logic                                   wr ;
logic                                   wrDone ;
logic    [C_S_AXI_ADDR_WIDTH-1:0]       rdAddr ;
logic    [C_S_AXI_DATA_WIDTH-1:0]       rdData ;
logic                                   rd ;
logic                                   rdDone ;

// Data arrays
logic signed [RAM_DATA_WIDTH-1:0]       coeffs[NUM_TAPS-1:0][FIR_COPIES-1:0];
logic signed [RAM_DATA_WIDTH-1:0]       sin_L[NUM_SAMPLES-1:0];
logic signed [RAM_DATA_WIDTH-1:0]       sin_R[NUM_SAMPLES-1:0];

logic signed [RAM_DATA_WIDTH-1:0]       result_L[NUM_SAMPLES-1:0];
logic signed [RAM_DATA_WIDTH-1:0]       result_R[NUM_SAMPLES-1:0];

// Simulation Variables
logic macStatus; // Taken from LSB of rdData when reading from macStatus FIR
    filter signal.
int i;
int j;

logic signed [C_S_AXI_DATA_WIDTH-1:0] attenFactors_TB [0:FIR_COPIES-1];

//DUTs
FIR_Filter #(.C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH),.C_S_AXI_ADDR_WIDTH(
    C_S_AXI_ADDR_WIDTH),
              .RAM_DATA_WIDTH(RAM_DATA_WIDTH),.RAM_ADDR_WIDTH(RAM_ADDR_WIDTH))
                 FIR_Filter1 (
         // Axi4LiteRegs Bus
        .S_AXI_ACLK(S_AXI_ACLK),            // input
        .S_AXI_ARESETN(S_AXI_ARESETN),      // input
        .S_AXI_AWADDR(S_AXI_AWADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_AWVALID(S_AXI_AWVALID),      // input
        .S_AXI_AWREADY(S_AXI_AWREADY),      // output
        .S_AXI_WDATA(S_AXI_WDATA),          // input    [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_WSTRB(S_AXI_WSTRB),          // input    [3:0]
        .S_AXI_WVALID(S_AXI_WVALID),        // input
        .S_AXI_WREADY(S_AXI_WREADY),        // output
        .S_AXI_ARADDR(S_AXI_ARADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_ARVALID(S_AXI_ARVALID),      // input
        .S_AXI_ARREADY(S_AXI_ARREADY),      // output
        .S_AXI_RDATA(S_AXI_RDATA),          // output   [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_RRESP(S_AXI_RRESP),          // output   [1:0]
        .S_AXI_RVALID(S_AXI_RVALID),        // output
```

```
        .S_AXI_RREADY(S_AXI_RREADY),        // input
        .S_AXI_BRESP(S_AXI_BRESP),          // output   [1:0]
        .S_AXI_BVALID(S_AXI_BVALID),        // output
        .S_AXI_BREADY(S_AXI_BREADY),         // input
        .attenFactors(attenFactors_TB)
);
Axi4LiteManager #(.C_M_AXI_ADDR_WIDTH(C_S_AXI_ADDR_WIDTH), .C_M_AXI_DATA_WIDTH(
    C_S_AXI_DATA_WIDTH)) Axi4LiteManager1
        (
        // Simple Bus
        .wrAddr(wrAddr),                    // input    [C_M_AXI_ADDR_WIDTH-1:0]
        .wrData(wrData),                    // input    [C_M_AXI_DATA_WIDTH-1:0]
        .wr(wr),                            // input
        .wrDone(wrDone),                    // output
        .rdAddr(rdAddr),                    // input    [C_M_AXI_ADDR_WIDTH-1:0]
        .rdData(rdData),                    // output   [C_M_AXI_DATA_WIDTH-1:0]
        .rd(rd),                            // input
        .rdDone(rdDone),                    // output
        // Axi4Lite Bus
        .M_AXI_ACLK(S_AXI_ACLK),            // input
        .M_AXI_ARESETN(S_AXI_ARESETN),      // input
        .M_AXI_AWADDR(S_AXI_AWADDR),        // output   [C_M_AXI_ADDR_WIDTH-1:0]
        .M_AXI_AWVALID(S_AXI_AWVALID),      // output
        .M_AXI_AWREADY(S_AXI_AWREADY),      // input
        .M_AXI_WDATA(S_AXI_WDATA),          // output   [C_M_AXI_DATA_WIDTH-1:0]
        .M_AXI_WSTRB(S_AXI_WSTRB),          // output   [3:0]
        .M_AXI_WVALID(S_AXI_WVALID),        // output
        .M_AXI_WREADY(S_AXI_WREADY),        // input
        .M_AXI_ARADDR(S_AXI_ARADDR),        // output   [C_M_AXI_ADDR_WIDTH-1:0]
        .M_AXI_ARVALID(S_AXI_ARVALID),      // output
        .M_AXI_ARREADY(S_AXI_ARREADY),      // input
        .M_AXI_RDATA(S_AXI_RDATA),          // input    [C_M_AXI_DATA_WIDTH-1:0]
        .M_AXI_RRESP(S_AXI_RRESP),          // input    [1:0]
        .M_AXI_RVALID(S_AXI_RVALID),        // input
        .M_AXI_RREADY(S_AXI_RREADY),        // output
        .M_AXI_BRESP(S_AXI_BRESP),          // input    [1:0]
        .M_AXI_BVALID(S_AXI_BVALID),        // input
        .M_AXI_BREADY(S_AXI_BREADY)         // output
        );

    parameter CLK_PERIOD_2 = (CLK_PERIOD)/2;
    // Generate Clock
    always begin
        #(CLK_PERIOD_2) S_AXI_ACLK = ~S_AXI_ACLK;
```

```
end

initial begin
    // Initialize
    S_AXI_ARESETN = 0;
    S_AXI_ACLK = 0;
    wr = 0;
    wrAddr = 0;
    wrData = 0;
    rd = 0;
    rdAddr = 0;

    // Read filter coefficents and sine waves into arrays.
    $readmemh("Sin_1318_Hex.txt", sin_L);
    $readmemh("Sin_1734_Hex.txt", sin_R);
    $readmemh("Coeffs_Hex.mem", coeffs);
    $readmemb("attenFactors_2_14.mem", attenFactors_TB);

    //Generate Reset
    #(CLK_PERIOD_2 + 2) S_AXI_ARESETN = 1;
    #(CLK_PERIOD*10);


    // Fill coeff buffer
    for(j = 0; j < FIR_COPIES; j++) begin
        wrAddr = 4 * j; //Address of coeff RAM
        for(i = NUM_TAPS - 1; i >= 0; i--)begin

            if(j < 2) begin
                wrData = 0;
                wrAddr = 'h34 + 4*j; //Address of buffer RAM
                wr = 1;
                #(CLK_PERIOD*2);
                wr = 0;
                while(!wrDone) begin
                    #(CLK_PERIOD);
                end
                wrAddr = 4 * j; //Address of coeff RAM
            end

            wrData = coeffs[i][j];
            wr = 1;
            #(CLK_PERIOD*2);
            wr = 0;
```

```
        while(!wrDone) begin
            #(CLK_PERIOD);
        end
    end
end

#(CLK_PERIOD*10);

//Enable FIR
wrAddr = 'h50;
wrData = 1;
wr = 1;
#(CLK_PERIOD*2);
wr = 0;
while(!wrDone) begin
    #(CLK_PERIOD);
end



for(i = 0; i < NUM_SAMPLES; i++) begin
    for(int c = 0; c < 2; c++) begin
        if(c == 0) begin
            wrAddr = 'h34; // Left channel
            wrData = sin_L[i];
        end
        else begin
            wrAddr = 'h38; // Right channel
            wrData = sin_R[i];
        end

        wr = 1;
        #(CLK_PERIOD*2);
        wr = 0;

        while(!wrDone) begin
            #(CLK_PERIOD*1);
        end

        // Read macStatus
        rdAddr='h44; // Mac flag
        rd = 1;
        #(CLK_PERIOD*2);
        rd = 0;
```

```
            while(!rdDone) begin
                #(CLK_PERIOD*1);
            end
            macStatus = rdData[0];

            // keep reading macStatus until the MAC is complete
            while(macStatus)begin
                rd = 1;
                #(CLK_PERIOD*2);
                rd = 0;
                while(!rdDone) begin
                    #(CLK_PERIOD*1);
                end
                macStatus = rdData[0];
            end

            //Result Address
            if(c == 0)
                rdAddr = 'h3c;
            else
                rdAddr = 'h40;

            rd = 1;
            #(CLK_PERIOD*2);
            rd = 0;

            while(!rdDone) begin
                #(CLK_PERIOD*1);
            end

            if(c == 0)
                result_L[i] = rdData;
            else
                result_R[i] = rdData;

        end
    end

    $display("Left Channel results: \n");
    for(i = 0; i < NUM_SAMPLES; i++) begin
        $display("%d \n", result_L[i]);
    end

    $display("Right Channel results: \n");
```

```
        for(i = 0; i < NUM_SAMPLES; i++) begin
            $display("%d \n", result_R[i]);
        end
        $stop;
    end
endmodule
```

Listing 3: FIR filter SV testbench.

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////
// Engineers: Elias Dahl, Andres Reis
// Module Name: Audio_EQ
////////////////////////////////////////////////////////////////////////////////


module Audio_EQ_Final#(parameter C_S_AXI_DATA_WIDTH = 32, C_S_AXI_ADDR_WIDTH = 9,
    RAM_DATA_WIDTH = 16, RAM_ADDR_WIDTH = 9, FIR_COPIES = 13, NUM_CHANNELS = 2,
    NUM_TAPS = 279)
(
    /*Supporter Signals*/
    input  logic  S_AXI_ACLK,
    input  logic  S_AXI_ARESETN,
    input  logic  [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR,
    input  logic  S_AXI_AWVALID,
    output logic  S_AXI_AWREADY,
    input  logic  [C_S_AXI_DATA_WIDTH-1:0] S_AXI_WDATA,
    input  logic  [3:0] S_AXI_WSTRB,
    input  logic  S_AXI_WVALID,
    output logic  S_AXI_WREADY,
    input  logic  [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_ARADDR,
    input  logic  S_AXI_ARVALID,
    output logic  S_AXI_ARREADY,
    output logic  [C_S_AXI_DATA_WIDTH-1:0] S_AXI_RDATA,
    output logic  [1:0] S_AXI_RRESP,
    output logic  S_AXI_RVALID,
    input  logic  S_AXI_RREADY,
    output logic  [1:0] S_AXI_BRESP,
    output logic  S_AXI_BVALID,
    input  logic  S_AXI_BREADY,

    /*SPI Signals: 0 -> ADC; 1 -> DAC*/
    output logic  [1:0]SCK,
    output logic  [1:0]SDI,
    output logic  [1:0]CONV,
    input  logic  [1:0]SDO
    );

    /*Internal Manager Signals. 0 -> ADC SPI; 1 -> DAC SPI; 2 -> FIR*/
    logic   [C_S_AXI_ADDR_WIDTH-1:0] M_AXI_AWADDR [0:2];
    logic   [2:0] M_AXI_AWVALID ;
    logic   [2:0] M_AXI_AWREADY ;
    logic   [C_S_AXI_DATA_WIDTH-1:0] M_AXI_WDATA [0:2] ;
```

```
logic   [3:0] M_AXI_WSTRB [0:2] ;
logic   [2:0] M_AXI_WVALID ;
logic   [2:0] M_AXI_WREADY ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] M_AXI_ARADDR [0:2] ;
logic   [2:0] M_AXI_ARVALID ;
logic   [2:0] M_AXI_ARREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] M_AXI_RDATA [0:2] ;
logic   [1:0] M_AXI_RRESP [0:2] ;
logic   [2:0] M_AXI_RVALID ;
logic   [2:0] M_AXI_RREADY ;
logic   [1:0] M_AXI_BRESP [0:2] ;
logic   [2:0] M_AXI_BVALID ;
logic   [2:0] M_AXI_BREADY ;

// Manager Simple Busses
logic   [C_S_AXI_ADDR_WIDTH-1:0]    M_wrAddr [0:2] ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    M_wrData [0:2] ;
logic                       [2:0]   M_wr ;
logic                       [2:0]   M_wrDone ;
logic   [C_S_AXI_ADDR_WIDTH-1:0]    M_rdAddr [0:2] ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    M_rdData [0:2] ;
logic                       [2:0]   M_rd ;
logic                       [2:0]   M_rdDone ;

/*Supporter simple bus*/
logic   [C_S_AXI_ADDR_WIDTH-1:0]    S_wrAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    S_wrData ;
logic                               S_wr ;
logic   [C_S_AXI_ADDR_WIDTH-1:0]    S_rdAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    S_rdData ;
logic                               S_rd ;

/*Attenuation Frequnecy Flops*/
logic signed [C_S_AXI_DATA_WIDTH-1:0] attenFactors_D[0:FIR_COPIES-1];
logic signed [C_S_AXI_DATA_WIDTH-1:0] attenFactors_Q[0:FIR_COPIES-1];

/*EQ Enable*/
logic EQ_Enable_D, EQ_Enable_Q;

logic channelSelect_D, channelSelect_Q;

logic RWCounter_D, RWCounter_Q;

logic macStatus_D, macStatus_Q;
```

```
/* Sample Flops */
logic   [C_S_AXI_DATA_WIDTH-1:0] currSample_D, currSample_Q;

/*FSM States*/
typedef enum logic [4:0] {INIT_0, INIT_1, INIT_2, INIT_3, IDLE, READ_ADC_0,
    READ_ADC_1,
            WRITE_FIR_0, WRITE_FIR_1, READ_FIR_0, READ_FIR_1, WRITE_SPIS_0,
                WRITE_SPIS_1, WAIT_DAC} statetype;

statetype nextState, currState;

/*Internal Devices*/

// Axi4Lite Supporter Bus, used for communicating with the Microblaze/ EQ
    Program.
Axi4LiteSupporter #(.C_S_AXI_ADDR_WIDTH(C_S_AXI_ADDR_WIDTH),.
    C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH)) Supporter_0 (
    // Simple Bus
    .wrAddr(S_wrAddr),                      // output   [C_S_AXI_ADDR_WIDTH-1:0]
    .wrData(S_wrData),                      // output   [C_S_AXI_DATA_WIDTH-1:0]
    .wr(S_wr),                              // output
    .rdAddr(S_rdAddr),                      // output   [C_S_AXI_ADDR_WIDTH-1:0]
    .rdData(S_rdData),                      // input    [C_S_AXI_ADDR_WIDTH-1:0]
    .rd(S_rd),                              // output
    // Axi4Lite Bus
    .S_AXI_ACLK(S_AXI_ACLK),            // input
    .S_AXI_ARESETN(S_AXI_ARESETN),      // input
    .S_AXI_AWADDR(S_AXI_AWADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
    .S_AXI_AWVALID(S_AXI_AWVALID),      // input
    .S_AXI_AWREADY(S_AXI_AWREADY),      // output
    .S_AXI_WDATA(S_AXI_WDATA),          // input    [C_S_AXI_DATA_WIDTH-1:0]
    .S_AXI_WSTRB(S_AXI_WSTRB),          // input    [3:0]
    .S_AXI_WVALID(S_AXI_WVALID),        // input
    .S_AXI_WREADY(S_AXI_WREADY),        // output
    .S_AXI_ARADDR(S_AXI_ARADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
    .S_AXI_ARVALID(S_AXI_ARVALID),      // input
    .S_AXI_ARREADY(S_AXI_ARREADY),      // output
    .S_AXI_RDATA(S_AXI_RDATA),          // output   [C_S_AXI_DATA_WIDTH-1:0]
    .S_AXI_RRESP(S_AXI_RRESP),          // output   [1:0]
    .S_AXI_RVALID(S_AXI_RVALID),        // output
    .S_AXI_RREADY(S_AXI_RREADY),        // input
    .S_AXI_BRESP(S_AXI_BRESP),          // output   [1:0]
```

```
    .S_AXI_BVALID(S_AXI_BVALID),        // output
    .S_AXI_BREADY(S_AXI_BREADY)         // input
    ) ;

FIR_Filter #(.C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH), .C_S_AXI_ADDR_WIDTH(
    C_S_AXI_ADDR_WIDTH)) FIR_0 (
 // Axi4LiteRegs Bus
    .S_AXI_ACLK(S_AXI_ACLK),            // input
    .S_AXI_ARESETN(S_AXI_ARESETN),      // input
    .S_AXI_AWADDR(M_AXI_AWADDR[2]),        // input    [C_S_AXI_ADDR_WIDTH
        -1:0]
    .S_AXI_AWVALID(M_AXI_AWVALID[2]),      // input
    .S_AXI_AWREADY(M_AXI_AWREADY[2]),      // output
    .S_AXI_WDATA(M_AXI_WDATA[2]),          // input    [C_S_AXI_DATA_WIDTH
        -1:0]
    .S_AXI_WSTRB(M_AXI_WSTRB[2]),          // input    [3:0]
    .S_AXI_WVALID(M_AXI_WVALID[2]),        // input
    .S_AXI_WREADY(M_AXI_WREADY[2]),        // output
    .S_AXI_ARADDR(M_AXI_ARADDR[2]),        // input    [C_S_AXI_ADDR_WIDTH
        -1:0]
    .S_AXI_ARVALID(M_AXI_ARVALID[2]),      // input
    .S_AXI_ARREADY(M_AXI_ARREADY[2]),      // output
    .S_AXI_RDATA(M_AXI_RDATA[2]),          // output   [C_S_AXI_DATA_WIDTH
        -1:0]
    .S_AXI_RRESP(M_AXI_RRESP[2]),          // output   [1:0]
    .S_AXI_RVALID(M_AXI_RVALID[2]),        // output
    .S_AXI_RREADY(M_AXI_RREADY[2]),        // input
    .S_AXI_BRESP(M_AXI_BRESP[2]),          // output   [1:0]
    .S_AXI_BVALID(M_AXI_BVALID[2]),        // output
    .S_AXI_BREADY(M_AXI_BREADY[2]),         // input
    .attenFactors(attenFactors_Q),          // input
    .JA_1(macStatus_D)                      // output
);

genvar j;
generate
    /* Managers */
    for(j = 0; j < 3; j++) begin
        Axi4LiteManager #(.C_M_AXI_ADDR_WIDTH(C_S_AXI_ADDR_WIDTH), .
            C_M_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH)) Manager_
        (
        // Simple Bus
        .wrAddr(M_wrAddr[j]),                    // input    [
            C_M_AXI_ADDR_WIDTH-1:0]
```

```
        .wrData(M_wrData[j]),                       // input    [
            C_M_AXI_DATA_WIDTH-1:0]
        .wr(M_wr[j]),                               // input
        .wrDone(M_wrDone[j]),                       // output
        .rdAddr(M_rdAddr[j]),                       // input    [
            C_M_AXI_ADDR_WIDTH-1:0]
        .rdData(M_rdData[j]),                       // output   [
            C_M_AXI_DATA_WIDTH-1:0]
        .rd(M_rd[j]),                               // input
        .rdDone(M_rdDone[j]),                       // output
        // Axi4Lite Bus
        .M_AXI_ACLK(S_AXI_ACLK),              // input
        .M_AXI_ARESETN(S_AXI_ARESETN),        // input
        .M_AXI_AWADDR(M_AXI_AWADDR[j]),          // output   [C_M_AXI_ADDR_WIDTH
            -1:0]
        .M_AXI_AWVALID(M_AXI_AWVALID[j]),        // output
        .M_AXI_AWREADY(M_AXI_AWREADY[j]),        // input
        .M_AXI_WDATA(M_AXI_WDATA[j]),            // output   [C_M_AXI_DATA_WIDTH
            -1:0]
        .M_AXI_WSTRB(M_AXI_WSTRB[j]),            // output   [3:0]
        .M_AXI_WVALID(M_AXI_WVALID[j]),          // output
        .M_AXI_WREADY(M_AXI_WREADY[j]),          // input
        .M_AXI_ARADDR(M_AXI_ARADDR[j]),          // output   [C_M_AXI_ADDR_WIDTH
            -1:0]
        .M_AXI_ARVALID(M_AXI_ARVALID[j]),        // output
        .M_AXI_ARREADY(M_AXI_ARREADY[j]),        // input
        .M_AXI_RDATA(M_AXI_RDATA[j]),            // input    [C_M_AXI_DATA_WIDTH
            -1:0]
        .M_AXI_RRESP(M_AXI_RRESP[j]),            // input    [1:0]
        .M_AXI_RVALID(M_AXI_RVALID[j]),          // input
        .M_AXI_RREADY(M_AXI_RREADY[j]),          // output
        .M_AXI_BRESP(M_AXI_BRESP[j]),            // input    [1:0]
        .M_AXI_BVALID(M_AXI_BVALID[j]),          // input
        .M_AXI_BREADY(M_AXI_BREADY[j])           // output
        );
    end

    /* SPIs */
    for (j = 0; j < 2; j++) begin
        SPI #(.C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH),.C_S_AXI_ADDR_WIDTH(
            C_S_AXI_ADDR_WIDTH)) SPI_ (
         // Axi4LiteRegs Bus
        .S_AXI_ACLK(S_AXI_ACLK),              // input
        .S_AXI_ARESETN(S_AXI_ARESETN),        // input
```

```
        .S_AXI_AWADDR(M_AXI_AWADDR[j]),         // input    [C_S_AXI_ADDR_WIDTH
            -1:0]
        .S_AXI_AWVALID(M_AXI_AWVALID[j]),       // input
        .S_AXI_AWREADY(M_AXI_AWREADY[j]),       // output
        .S_AXI_WDATA(M_AXI_WDATA[j]),           // input    [C_S_AXI_DATA_WIDTH
            -1:0]
        .S_AXI_WSTRB(M_AXI_WSTRB[j]),           // input    [3:0]
        .S_AXI_WVALID(M_AXI_WVALID[j]),         // input
        .S_AXI_WREADY(M_AXI_WREADY[j]),         // output
        .S_AXI_ARADDR(M_AXI_ARADDR[j]),         // input    [C_S_AXI_ADDR_WIDTH
            -1:0]
        .S_AXI_ARVALID(M_AXI_ARVALID[j]),       // input
        .S_AXI_ARREADY(M_AXI_ARREADY[j]),       // output
        .S_AXI_RDATA(M_AXI_RDATA[j]),           // output   [C_S_AXI_DATA_WIDTH
            -1:0]
        .S_AXI_RRESP(M_AXI_RRESP[j]),           // output   [1:0]
        .S_AXI_RVALID(M_AXI_RVALID[j]),         // output
        .S_AXI_RREADY(M_AXI_RREADY[j]),         // input
        .S_AXI_BRESP(M_AXI_BRESP[j]),           // output   [1:0]
        .S_AXI_BVALID(M_AXI_BVALID[j]),         // output
        .S_AXI_BREADY(M_AXI_BREADY[j]),          // input
        .SCK(SCK[j]),
        .SDI(SDI[j]),
        .CONV(CONV[j]),
        .SDO(SDO[j])
        );
    end
endgenerate

    /* Sequential Logic */
always_ff @ (posedge S_AXI_ACLK) begin
    if(!S_AXI_ARESETN) begin
        currState <= INIT_0;
        currSample_Q <= 0;
        EQ_Enable_Q <= 0;
        channelSelect_Q <= 0;
        RWCounter_Q <= 0;
        macStatus_Q <= 0;
        for(int i = 0; i < FIR_COPIES; i++) begin
            attenFactors_Q[i] <= 0;
        end
    end
    else begin
        currState <= nextState;
```

```
        currSample_Q <= currSample_D;
        EQ_Enable_Q <= EQ_Enable_D;
        channelSelect_Q <= channelSelect_D;
        RWCounter_Q <= RWCounter_D;
        macStatus_Q <= macStatus_D;
        for(int i = 0; i < FIR_COPIES; i++) begin
            attenFactors_Q[i] <= attenFactors_D[i];
        end
    end
end

/* Combinational Logic */
always_comb begin
    /* Default Values */

    // Flops
    nextState = currState;
    currSample_D = currSample_Q;
    EQ_Enable_D = EQ_Enable_Q;
    channelSelect_D = channelSelect_Q;
    RWCounter_D = RWCounter_Q;

    for(int i = 0; i < FIR_COPIES; i++) begin
        attenFactors_D[i] = attenFactors_Q[i];
    end

    // Manager Simple Bus
    for (int i = 0; i < 3; i++ )begin
        M_wrAddr[i] = 0;
        M_wrData[i] = 0;
        M_rdAddr[i] = 0;
    end

    M_wr = 0;
    M_rd = 0;

    // Supp simple bus
    S_rdData = 0;


    /* FSM */
    case(currState)
        /* When the EQ is enabled, intiates a write of the number of bits used
            to both SPIS. */
```

```
INIT_0: begin
    if(EQ_Enable_Q == 1) begin

        //Write to SPIs
        M_wrAddr[0] = 'h0; //Addr of num bits
        M_wrAddr[1] = 'h0; //Addr of num bits

        M_wrData[0] = 'd16; //Bits used by ADC SPI
        M_wrData[1] = 'd24; //Bits used by DAC SPI

        M_wr[0] = 1;
        M_wr[1] = 1;

        if(RWCounter_Q == 0) begin
            RWCounter_D = 1;
            nextState = INIT_0;
        end
        else begin
            RWCounter_D = 0;
            nextState = INIT_1;
        end
    end
end

/* Waits for the SPI writes to complete */
INIT_1: begin
    if(M_wrDone[0] == 1 && M_wrDone[1] == 1) begin
        nextState = INIT_2;
    end
    else begin
        nextState = INIT_1;
    end
end

/* Initiates a write for the number of cycles used by both SPIS, and
   enables the FIR filter.*/
INIT_2: begin
    //Write to SPIs

    M_wrAddr[0] = 'h10; //Addr of num cycles
    M_wrAddr[1] = 'h10; //Addr of num cycles
    M_wrAddr[2] = 'h50; // FIR Enable

    M_wrData[0] = 'd300; // SPI Cycles
```

```
        M_wrData[1] = 'd300; // SPI Cycles
        M_wrData[2] = 1;

        M_wr[0] = 1;
        M_wr[1] = 1;
        M_wr[2] = 1;

        if(RWCounter_Q == 0) begin
            RWCounter_D = 1;
            nextState = INIT_2;
        end
        else begin
            RWCounter_D = 0;
            nextState = INIT_3;
        end

    end

    /* Wait for the writes to complete. */
    INIT_3: begin
        if(M_wrDone[0] == 1 && M_wrDone[1] == 1) begin
            nextState = IDLE;
        end
        else begin
            nextState = INIT_3;
        end
    end

    /* Waits for the last DAC cycle to complete before starting the next
        loop.*/
    IDLE: begin
        if(CONV[1] == 1) begin
            nextState = READ_ADC_0;
        end
        else begin
            nextState = IDLE;
        end
    end

    /* Initiates a read of the ADC output */
    READ_ADC_0: begin
        M_rdAddr[0] = 'h8; // ADC output data
        M_rd[0] = 1;
```

```
        if(RWCounter_Q == 0) begin
            RWCounter_D = 1;
            nextState = READ_ADC_0;
        end
        else begin
            RWCounter_D = 0;
            nextState = READ_ADC_1;
        end
    end

    /* Waits for the ADC read to complete, and puts the sample into
        currSample. */
    READ_ADC_1: begin
        if(M_rdDone[0] == 1) begin
            currSample_D = {'0,~(M_rdData[0][15]),M_rdData[0][14:0]}; //
                Unsigned -> Signed by flip "MSB"
            nextState = WRITE_FIR_0;
        end
        else begin
            nextState = READ_ADC_1;
        end
    end

    /* Writes a sample to the FIR filter.*/
    WRITE_FIR_0: begin
        if(channelSelect_Q == 0) begin
            M_wrAddr[2] = 'h38; // R channel
        end
        else begin
            M_wrAddr[2] = 'h34; // L channel
        end

        M_wrData[2] = currSample_Q;
        M_wr[2] = 1;

        if(RWCounter_Q == 0) begin
            RWCounter_D = 1;
            nextState = WRITE_FIR_0;
        end
        else begin
            RWCounter_D = 0;
            nextState = WRITE_FIR_1;
        end
    end
```

```
WRITE_FIR_1: begin
  if(M_wrDone[2] == 1) begin
        nextState = READ_FIR_0;
    end
    else begin
        nextState = WRITE_FIR_1;
    end
end


// Reads the previous FIR result
READ_FIR_0: begin
    if(channelSelect_Q == 0) begin
        M_rdAddr[2] = 'h40; // R channel
    end
    else begin
        M_rdAddr[2] = 'h3c; // L channel
    end


    M_rd[2] = 1;

    if(RWCounter_Q == 0) begin
        RWCounter_D = 1;
        nextState = READ_FIR_0;
    end
    else begin
        RWCounter_D = 0;
        nextState = READ_FIR_1;
    end

end

// Waits for the FIR read to compete.
// Puts the FIR output in currSample when read completes.
READ_FIR_1: begin

    if(M_rdDone[2] == 1) begin
        currSample_D = {'0,~(M_rdData[2][15]),M_rdData[2][14:0]}; //
            signed -> Unsigned by flip "MSB"
        nextState = WRITE_SPIS_0;
    end
    else begin
```

```
        nextState = READ_FIR_1;
    end
end

/* Initates a write of the current sample to the DAC, and the SDI
   address data for the next ADC cycle. */
WRITE_SPIS_0: begin
    // Only address bits are different.
    if(channelSelect_Q == 0) begin
        M_wrData[1] = {'0,4'b0011,4'b0000,currSample_Q[15:0]}; // L
            channel
        M_wrData[0] = 'hc000; // ch1 to be processed next next?
    end
    else begin
        M_wrData[1] = {'0,4'b0011,4'b0001,currSample_Q[15:0]}; // R
            channel
        M_wrData[0] = 'h8000; // ch0 to be processed next next?
    end

    M_wrAddr[1] = 'h4; // SDI
    M_wr[1] = 1;

    M_wrAddr[0] = 'h4; // ADC SDI
    M_wr[0] = 1;

    if(RWCounter_Q == 0) begin
        RWCounter_D = 1;
        nextState = WRITE_SPIS_0;
    end
    else begin
        RWCounter_D = 0;
        channelSelect_D = (channelSelect_Q == 1) ? 0 : 1; //Swap
            channel for next pass.
        nextState = WRITE_SPIS_1;
    end

end

/* Waits for the writes to complete */
WRITE_SPIS_1: begin
    if(M_wrDone[1] == 1 && M_wrDone[0]) begin
        nextState = WAIT_DAC;
    end
    else begin
```

```
                nextState = WRITE_SPIS_1;
            end
        end

        /* Waits for the DAC SPI CONV to go low so IDLE can wait for it to go
            high. */
        WAIT_DAC: begin
            if(CONV[1] == 1) begin
                nextState = WAIT_DAC;
            end
            else begin
            nextState = IDLE;
            end
        end

        default: begin
            nextState = INIT_0;
        end
    endcase

    if(S_wr) begin
        /* Writes to the FIR coefficient buffers */
        /* Addresses from 0x0 to 0x30 */
        for(int i=0;i<FIR_COPIES;i++) begin
            if(S_wrAddr == (4 * i)) begin
                M_wrData[2] = S_wrData;
                M_wrAddr[2] = S_wrAddr;
                M_wr[2] = S_wr;
            end
        end

        /*Writes to the FIR sample buffers, used for initializing them to 0.*/
        /* 0x34-0x38 */
        for(int i=0; i < NUM_CHANNELS; i++) begin
            if(S_wrAddr == 'h34 + 4 * i) begin
                M_wrData[2] = S_wrData;
                M_wrAddr[2] = S_wrAddr;
                M_wr[2] = S_wr;
            end
        end

        /* Used to start the equalizer */
        if(S_wrAddr == 'h3c) begin
            EQ_Enable_D = S_wrData[0];
```

```
            end

            /* Used to write to the attenuation factor flops. */
            /* 0x40 - 0x70 */
            for(int i = 0; i < FIR_COPIES; i++) begin
                if(S_wrAddr == 'h40 + 4*i) begin
                    attenFactors_D[i] = {'0,S_wrData[15:0]};
                end
            end
        end
    end
endmodule
```

Listing 4: Audio equalizer SV module.

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Engineer: Elias Dahl, Andres Reis
//
// Create Date: 10/02/2025 01:30:37 PM
// Design Name:
// Module Name: FIR_Filter_TB
// Project Name:
// Target Devices:
// Description:
//
// Dependencies:
//
//////////////////////////////////////////////////////////////////////////////////


module EQ_TB();
parameter C_S_AXI_DATA_WIDTH = 32, C_S_AXI_ADDR_WIDTH = 9, CLK_PERIOD = 33.33,
    RAM_DATA_WIDTH = 16, RAM_ADDR_WIDTH = 9, NUM_TAPS = 279, NUM_SAMPLES = 1000,
    FIR_COPIES = 13;
// Axi4Lite signals
logic   S_AXI_ACLK ;
logic   S_AXI_ARESETN ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR ;
logic   S_AXI_AWVALID ;
logic   S_AXI_AWREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] S_AXI_WDATA ;
logic   [3:0] S_AXI_WSTRB ;
logic   S_AXI_WVALID ;
logic   S_AXI_WREADY ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_ARADDR ;
logic   S_AXI_ARVALID ;
logic   S_AXI_ARREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] S_AXI_RDATA ;
logic   [1:0] S_AXI_RRESP ;
logic   S_AXI_RVALID ;
logic   S_AXI_RREADY ;
logic   [1:0] S_AXI_BRESP ;
logic   S_AXI_BVALID ;
logic   S_AXI_BREADY ;

// SPI signals
logic  [1:0]SCK;
logic  [1:0]SDI;
```

```
logic  [1:0]CONV;
logic  [1:0]SDO;

// Manager simple Bus signals
logic   [C_S_AXI_ADDR_WIDTH-1:0]    wrAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    wrData ;
logic                               wr ;
logic                               wrDone ;
logic   [C_S_AXI_ADDR_WIDTH-1:0]    rdAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    rdData ;
logic                               rd ;
logic                               rdDone ;

// Data arrays
logic signed [RAM_DATA_WIDTH-1:0]   coeffs[NUM_TAPS-1:0][FIR_COPIES-1:0];

logic signed [RAM_DATA_WIDTH-1:0]   result_L[NUM_SAMPLES-1:0];
logic signed [RAM_DATA_WIDTH-1:0]   result_R[NUM_SAMPLES-1:0];

// Simulation Variables
logic macStatus; // Taken from LSB of rdData when reading from macStatus FIR
    filter signal.
int i;
int j;

//SPI Signals

logic signed [C_S_AXI_DATA_WIDTH-1:0] attenFactors_TB [0:FIR_COPIES-1];

//DUTs
Audio_EQ #(.C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH),.C_S_AXI_ADDR_WIDTH(
    C_S_AXI_ADDR_WIDTH),
              .RAM_DATA_WIDTH(RAM_DATA_WIDTH),.RAM_ADDR_WIDTH(RAM_ADDR_WIDTH))
                 EQ_DUT (
         // Axi4LiteRegs Bus
        .S_AXI_ACLK(S_AXI_ACLK),            // input
        .S_AXI_ARESETN(S_AXI_ARESETN),      // input
        .S_AXI_AWADDR(S_AXI_AWADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_AWVALID(S_AXI_AWVALID),      // input
        .S_AXI_AWREADY(S_AXI_AWREADY),      // output
        .S_AXI_WDATA(S_AXI_WDATA),          // input    [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_WSTRB(S_AXI_WSTRB),          // input    [3:0]
        .S_AXI_WVALID(S_AXI_WVALID),        // input
        .S_AXI_WREADY(S_AXI_WREADY),        // output
```

```
        .S_AXI_ARADDR(S_AXI_ARADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_ARVALID(S_AXI_ARVALID),      // input
        .S_AXI_ARREADY(S_AXI_ARREADY),      // output
        .S_AXI_RDATA(S_AXI_RDATA),          // output   [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_RRESP(S_AXI_RRESP),          // output   [1:0]
        .S_AXI_RVALID(S_AXI_RVALID),        // output
        .S_AXI_RREADY(S_AXI_RREADY),        // input
        .S_AXI_BRESP(S_AXI_BRESP),          // output   [1:0]
        .S_AXI_BVALID(S_AXI_BVALID),        // output
        .S_AXI_BREADY(S_AXI_BREADY),         // input
        .SCK(SCK),
        .SDO(SDO),
        .SDI(SDI),
        .CONV(CONV)
);
Axi4LiteManager #(.C_M_AXI_ADDR_WIDTH(C_S_AXI_ADDR_WIDTH), .C_M_AXI_DATA_WIDTH(
    C_S_AXI_DATA_WIDTH)) Manager_TB
        (
        // Simple Bus
        .wrAddr(wrAddr),                    // input    [C_M_AXI_ADDR_WIDTH-1:0]
        .wrData(wrData),                    // input    [C_M_AXI_DATA_WIDTH-1:0]
        .wr(wr),                            // input
        .wrDone(wrDone),                    // output
        .rdAddr(rdAddr),                    // input    [C_M_AXI_ADDR_WIDTH-1:0]
        .rdData(rdData),                    // output   [C_M_AXI_DATA_WIDTH-1:0]
        .rd(rd),                            // input
        .rdDone(rdDone),                    // output
        // Axi4Lite Bus
        .M_AXI_ACLK(S_AXI_ACLK),            // input
        .M_AXI_ARESETN(S_AXI_ARESETN),      // input
        .M_AXI_AWADDR(S_AXI_AWADDR),        // output   [C_M_AXI_ADDR_WIDTH-1:0]
        .M_AXI_AWVALID(S_AXI_AWVALID),      // output
        .M_AXI_AWREADY(S_AXI_AWREADY),      // input
        .M_AXI_WDATA(S_AXI_WDATA),          // output   [C_M_AXI_DATA_WIDTH-1:0]
        .M_AXI_WSTRB(S_AXI_WSTRB),          // output   [3:0]
        .M_AXI_WVALID(S_AXI_WVALID),        // output
        .M_AXI_WREADY(S_AXI_WREADY),        // input
        .M_AXI_ARADDR(S_AXI_ARADDR),        // output   [C_M_AXI_ADDR_WIDTH-1:0]
        .M_AXI_ARVALID(S_AXI_ARVALID),      // output
        .M_AXI_ARREADY(S_AXI_ARREADY),      // input
        .M_AXI_RDATA(S_AXI_RDATA),          // input    [C_M_AXI_DATA_WIDTH-1:0]
        .M_AXI_RRESP(S_AXI_RRESP),          // input    [1:0]
        .M_AXI_RVALID(S_AXI_RVALID),        // input
        .M_AXI_RREADY(S_AXI_RREADY),        // output
```

```
    .M_AXI_BRESP(S_AXI_BRESP),          // input     [1:0]
    .M_AXI_BVALID(S_AXI_BVALID),        // input
    .M_AXI_BREADY(S_AXI_BREADY)         // output
    );

AdcTester AdcTester1 (
.SCK(SCK[0]),
.SDI(SDI[0]),
.CS_(CONV[0]),
.SDO(SDO[0])
);

DacTester DacTester1 (
.SCK(SCK[1]),
.SDI(SDI[1]),
.CS_(CONV[1]),
.SDO(SDO[1])
);

parameter CLK_PERIOD_2 = (CLK_PERIOD)/2;
// Generate Clock
always begin
    #(CLK_PERIOD_2) S_AXI_ACLK = ~S_AXI_ACLK;
end

initial begin
    // Initialize
    S_AXI_ARESETN = 0;
    S_AXI_ACLK = 0;
    wr = 0;
    wrAddr = 0;
    wrData = 0;
    rd = 0;
    rdAddr = 0;

    // Read filter coefficents and sine waves into arrays.
    $readmemh("Coeffs_Hex.mem", coeffs);

    //Generate Reset
    #(CLK_PERIOD_2 + 2) S_AXI_ARESETN = 1;
    #(CLK_PERIOD*10);


    // Fill coeff buffer
```

```
for(j = 0; j < FIR_COPIES; j++) begin
    wrAddr = 4 * j; //Address of coeff RAM
    for(i = NUM_TAPS - 1; i >= 0; i--)begin

        // init sample buffers to 0s
        if(j < 2) begin
            wrData = 0;
            wrAddr = 'h34 + 4*j; //Address of buffer RAM
            wr = 1;
            #(CLK_PERIOD*2);
            wr = 0;
            while(!wrDone) begin
                #(CLK_PERIOD);
            end
            wrAddr = 4 * j; //Address of coeff RAM
        end

        wrData = coeffs[i][j];
        wr = 1;
        #(CLK_PERIOD*2);
        wr = 0;
        while(!wrDone) begin
            #(CLK_PERIOD);
        end
    end
end

#(CLK_PERIOD*10);

for(i = 0; i < FIR_COPIES; i++) begin
    wrData = 'b0100_0000_0000_0000; // Attenuation of 1 in 2.14
    wrAddr = 'h40 + 4*i;

    wr = 1;
    #(CLK_PERIOD*2);
    wr = 0;

    while(!wrDone) begin
        #(CLK_PERIOD);
    end
end

#(CLK_PERIOD*10);
```

```
        //Enable EQ
        wrAddr = 'h3c; // ENABLE_EQ_ADDR
        wrData = 1;

        wr = 1;
        #(CLK_PERIOD*2);
        wr = 0;

        while(!wrDone) begin
            #(CLK_PERIOD);
        end

        $stop;
    end
endmodule
```

Listing 5: Audio equalizer SV testbench.

```c
/*****************************************************************************/
/**
 *
 * @authors
 * Elias Dahl
 * Andres Reis
 *
 * @file EQ_Comm.c
 *
 * This program initializes the hardware Audio equalizer by filling its coefficent
 * buffers and 0 initializing its sample buffers, then enabling it. Then, the
 * program waits to recieve attenuation factor data from the LabView GUI program,
 * writes the data to the audio equalizer, and waits for the next attenuation
 * factors. Modified from xuartlite_polled_example.c.

/***************************** Include Files *********************************/

#include "xparameters.h"
#include "xstatus.h"
#include "xuartlite.h"
#include "xil_printf.h"

#include "platform.h"
#include "xil_printf.h"

#include "coeffs.h"

/************************** Constant Definitions *****************************/

/*
 * The following constants map to the XPAR parameters created in the
 * xparameters.h file. They are defined here such that a user can easily
 * change all the needed parameters in one place.
 */
#ifndef SDT
#define UARTLITE_DEVICE_ID  XPAR_UARTLITE_0_DEVICE_ID
#else
#define XUARTLITE_BASEADDRESS XPAR_XUARTLITE_0_BASEADDR
#endif

/* Addresses */
#define EQ_BASE_ADDR 0x10000
#define BUFFER_LOW_ADDR (EQ_BASE_ADDR + 0x34)
#define EQ_ENABLE_ADDR (EQ_BASE_ADDR + 0x3c)
#define ATTEN_FACTORS_LOW_ADDR (EQ_BASE_ADDR + 0x40)

/*
 * The following constant controls the length of the buffers to be sent
 * and received with the UartLite, this constant must be 16 bytes or less since
 * this is a single threaded non-interrupt driven example such that the
 * entire buffer will fit into the transmit and receive FIFOs of the UartLite.
```

```
 */
#define TEST_BUFFER_SIZE 13*2


/************************** Function Prototypes ******************************/
#ifndef SDT
int UartLitePolledExample(u16 DeviceId);
#else
int UartLitePolledExample(UINTPTR BaseAddress);
#endif

/************************** Variable Definitions *****************************/

XUartLite UartLite; /* Instance of the UartLite Device */

u8 RecvBuffer[TEST_BUFFER_SIZE]; /* Buffer for Receiving Data */

/*****************************************************************************/
/**
 *
 * Main function to call the Uartlite polled example.
 *
 *
 * @return  XST_SUCCESS if successful, otherwise XST_FAILURE.
 *
 * @note    None.
 *
 ******************************************************************************/

int main(void) {
  init_platform();

  volatile int *wrAddr;

  int Status;

  /* Fills coeffs and init buffers to 0s. */
  for (int band = 0; band < NUM_BANDS; ++band) {

    wrAddr = (int *) (EQ_BASE_ADDR + 4 * band);

    for (int tap = 0; tap < NUM_TAPS; ++tap) {

      if (band < 2) {
        wrAddr = (int *) (BUFFER_LOW_ADDR + 4 * band);
        *wrAddr = 0;
        wrAddr = (int *) (EQ_BASE_ADDR + 4 * band);
      }

      *wrAddr = coeffs[band][tap];
    }
```

```
  }

  /* Init attenuation factors to 1*/
  for (int band = 0; band < NUM_BANDS; ++band) {

    wrAddr = (int *) (ATTEN_FACTORS_LOW_ADDR + 4 * band);
    *wrAddr = 0x4000; // 1 in 2.14
  }

  // Enable EQ
  wrAddr = (int *) EQ_ENABLE_ADDR;
  *wrAddr = 1;

  /*
   * Run the UartLite polled example, specify the Device ID that is
   * generated in xparameters.h
   */
#ifndef SDT
  Status = UartLitePolledExample(UARTLITE_DEVICE_ID);
#else
  Status = UartLitePolledExample(XUARTLITE_BASEADDRESS);
#endif
  if (Status != XST_SUCCESS) {
    xil_printf("Uartlite polled Example Failed\r\n");
    cleanup_platform();
    return XST_FAILURE;
  }

  xil_printf("Successfully ran Uartlite polled Example\r\n");
  cleanup_platform();
  return XST_SUCCESS;

}

/****************************************************************************/
/**
 * This function does a minimal test on the UartLite device and driver as a
 * design example. The purpose of this function is to illustrate
 * how to use the XUartLite component.
 *
 * This function sends data and expects to receive the data through the UartLite
 * such that a  physical loopback must be done with the transmit and receive
 * signals of the UartLite.
 *
 * This function polls the UartLite and does not require the use of interrupts.
 *
 * @param DeviceId is the Device ID of the UartLite and is the
 *    XPAR_<uartlite_instance>_DEVICE_ID value from xparameters.h.
 *
 * @return  XST_SUCCESS if successful, XST_FAILURE if unsuccessful.
 *
```

```
 *
 * @note
 *
 * This function calls the UartLite driver functions in a blocking mode such that
 * if the transmit data does not loopback to the receive, this function may
 * not return.
 *
 ****************************************************************************/
#ifndef SDT
int UartLitePolledExample(u16 DeviceId)
#else
int UartLitePolledExample(UINTPTR BaseAddress)
#endif
{
  int Status;
  unsigned int ReceivedCount = 0;
  int Index;

  /*
   * Initialize the UartLite driver so that it is ready to use.
   */
#ifndef SDT
  Status = XUartLite_Initialize(&UartLite, DeviceId);
#else
  Status = XUartLite_Initialize(&UartLite, BaseAddress);
#endif
  if (Status != XST_SUCCESS) {
    return XST_FAILURE;
  }

  /*
   * Perform a self-test to ensure that the hardware was built correctly.
   */
  Status = XUartLite_SelfTest(&UartLite);
  if (Status != XST_SUCCESS) {
    return XST_FAILURE;
  }

  /*
   * Initialize the receive buffer bytes to zero.
   */
  for (Index = 0; Index < TEST_BUFFER_SIZE; Index++) {
    RecvBuffer[Index] = 0;
  }

  /*
   * Receive the number of bytes which is transferred.
   * Data may be received in fifo with some delay hence we continuously
   * check the receive fifo for valid data and update the receive buffer
   * accordingly.
   */
```

```c
  volatile int *wrAddr;
  while (1) {
    ReceivedCount = 0;
    /* Wait for new attenuation factors to be sent*/
    while (1) {
      ReceivedCount += XUartLite_Recv(&UartLite,
          RecvBuffer + ReceivedCount,
          TEST_BUFFER_SIZE - ReceivedCount);
      if (ReceivedCount == TEST_BUFFER_SIZE) {
        break;
      }
    }

    /*Write the new attenuation factors*/
    for (int band = 0; band < NUM_BANDS; ++band) {

      wrAddr = (int *) (ATTEN_FACTORS_LOW_ADDR + 4 * band);
      int highByte = RecvBuffer[band * 2] << 8;
      int lowByte = RecvBuffer[(band * 2) + 1];
      *wrAddr = lowByte | highByte;
    }

  }

  return XST_SUCCESS;
}
```

Listing 6: Audio equalizer C driver program.